ANHANGUERA

MICHELA JULIANA OLIVEIRA

MATRICULA- 2025369608

RELATÓRIO DE AULA PRÁTICA

Unidade 2 – Estruturas Do Desenvolvimento Orientado A Objetos

SERTÃOZINHO -SP

2026

MICHELA JULIANA OLIVEIRA
MATRICULA- 2025369608

RELATÓRIO DE AULA PRÁTICA

Unidade 2 – Estruturas Do Desenvolvimento Orientado A Objetos

Trabalho de Aula Prática apresentado ao curso de Bacharelado em Engenharia de Software da Faculdade Anhanguera, como requisito parcial para obtenção de nota na disciplina de Linguagem Orientada A Objetos (Unidade 2 – Estruturas Do Desenvolvimento Orientado A Objetos).

SERTÃOZINHO -SP
2026

SUMARIO

1. INTRODUÇÃO ____ *4*

2. OBJETIVO ____ *5*

3. METODOLOGIA E DESENVOLVIMENTO ____ *6*

3.1 Execução do trabalho ____ *7*

4. RESULTADO ____ *9*

4.1 Arquivo Produto.java ____ *10*

4.2 Arquivo Principal.java ____ *11*

5. CONCLUSÃO ____ *12*

6. REFERENCIAS ____ *13*

1. INTRODUÇÃO

A Programação Orientada a Objetos (POO) é um paradigma de desenvolvimento de software essencial para a criação de sistemas modulares, escaláveis e de fácil manutenção. Esse modelo baseia-se na representação de entidades do mundo real por meio de objetos, os quais agrupam estados (atributos) e comportamentos (métodos) de forma estruturada e intuitiva.

No ecossistema da linguagem Java, o gerenciamento de instâncias e a alocação de recursos de memória desempenham papéis centrais no desempenho da aplicação. De acordo com Schildt (2015), os construtores são métodos especiais executados no momento de criação de um objeto, responsáveis por inicializar suas variáveis de instância. A aplicação da **sobrecarga de construtores** (*overloading*) permite que uma classe ofereça múltiplas formas de inicialização, conferindo maior flexibilidade e reuso de código durante o desenvolvimento.

Paralelamente, o uso do modificador static altera o escopo de variáveis e métodos, vinculando-os diretamente à classe e não às suas instâncias individuais. Segundo Silva, Leite e Oliveira (2019), membros estáticos são compartilhados por todos os objetos de uma mesma classe, tornando-se a solução ideal para implementar contadores globais e métodos utilitários que não dependem do estado de um objeto específico.

Este relatório apresenta a fundamentação teórica e o desenvolvimento prático de uma aplicação em Java construída no ambiente Eclipse IDE. O projeto simula o cadastro de produtos utilizando construtores padrão e parametrizados, integrados a um atributo e a um método estático para o controle exato e centralizado da quantidade total de itens cadastrados no sistema.

2. OBJETIVO

Este trabalho tem como objetivo aplicar os conceitos de construtores, sobrecarga e métodos estáticos no contexto da Programação Orientada a Objetos em Java. Por meio do desenvolvimento de uma aplicação de cadastro de produtos, demonstrou-se a criação de objetos via construtores padrão e parametrizados, bem como o uso de membros estáticos para gerenciar dados globais da classe. Dessa forma, consolidaram-se os princípios de instanciação e reutilização de código.

3. METODOLOGIA E DESENVOLVIMENTO

Para a execução da aula prática, utilizou-se a ferramenta Eclipse IDE for Java Developers.

Atividade Proposta: Desenvolver um programa em Java que simule o cadastro de produtos. O aluno deverá:

Criar a classe Produto com os seguintes atributos:

- String nome
- double preco
- static int quantidadeTotal (atributo compartilhado)

Implementar:

- Um construtor padrão e um construtor com parâmetros (sobrecarga);
- Um método exibirDados () para mostrar nome e preço;
- Um método estático exibirQuantidadeTotal () para mostrar quantos produtos foram cadastrados.

Na classe Principal:

- Criar ao menos três objetos Produto, usando os dois tipos de construtor;
- A cada novo produto, atualizar o atributo quantidadeTotal;
- Exibir os dados de cada produto;
- Exibir a quantidade total de produtos usando o método estático.

Procedimentos para a Realização da Atividade

PASSO 1 – Criar o Projeto Java no Eclipse

Abrir o Eclipse.

1. Ir em File > New > Java Project.
2. Nomear o projeto como CadastroProdutos.
3. Clicar em Finish.

PASSO 2 – Criar o Package

1. Clicar com o botão direito sobre src.

2. Selecionar New > Package.
3. Nomear como: br.edu.produto.

PASSO 3 – Criar a Classe Produto

1. Criar a classe com o nome Produto.
2. Declarar os seguintes atributos:

- String nome;
- double preco;
- static int quantidadeTotal = 0;

Criar dois construtores:

- Um construtor padrão que apenas incrementa quantidadeTotal.
- Um construtor com parâmetros que define nome e preco, e também incrementa quantidadeTotal.

1. Criar o método exibirDados() para imprimir nome e preço.
2. Criar o método estático exibirQuantidadeTotal() para imprimir o total de produtos cadastrados.

PASSO 4 – Criar a Classe Principal

1. Criar a classe Principal com o método main.
2. Instanciar três objetos da classe Produto, usando os dois tipos de construtor.
3. Chamar exibirDados() para cada produto.
4. Ao final, chamar Produto.exibirQuantidadeTotal().

Resumo do Que Deve Ser Feito: Crie a classe Produto com atributos, dois tipos de construtores (padrão e com parâmetros), um método para exibir os dados do produto e um método estático para mostrar a quantidade total de produtos criados. Em seguida, na classe Principal, instancie 3 produtos, usando os dois construtores, exiba os dados de cada um e mostre o total de produtos cadastrados.

3.1 Execução do trabalho

Para o desenvolvimento desta atividade prática na linguagem Java, aplicaram-se os conceitos fundamentais da Programação Orientada a Objetos (POO), com foco na organização estrutural, no encapsulamento e no gerenciamento de estado compartilhado. Inicialmente,

modelou-se a classe Produto para atuar como molde estrutural, contendo os atributos de estado individual nome (String) e preco (double), além do atributo compartilhado quantidadeTotal (int).

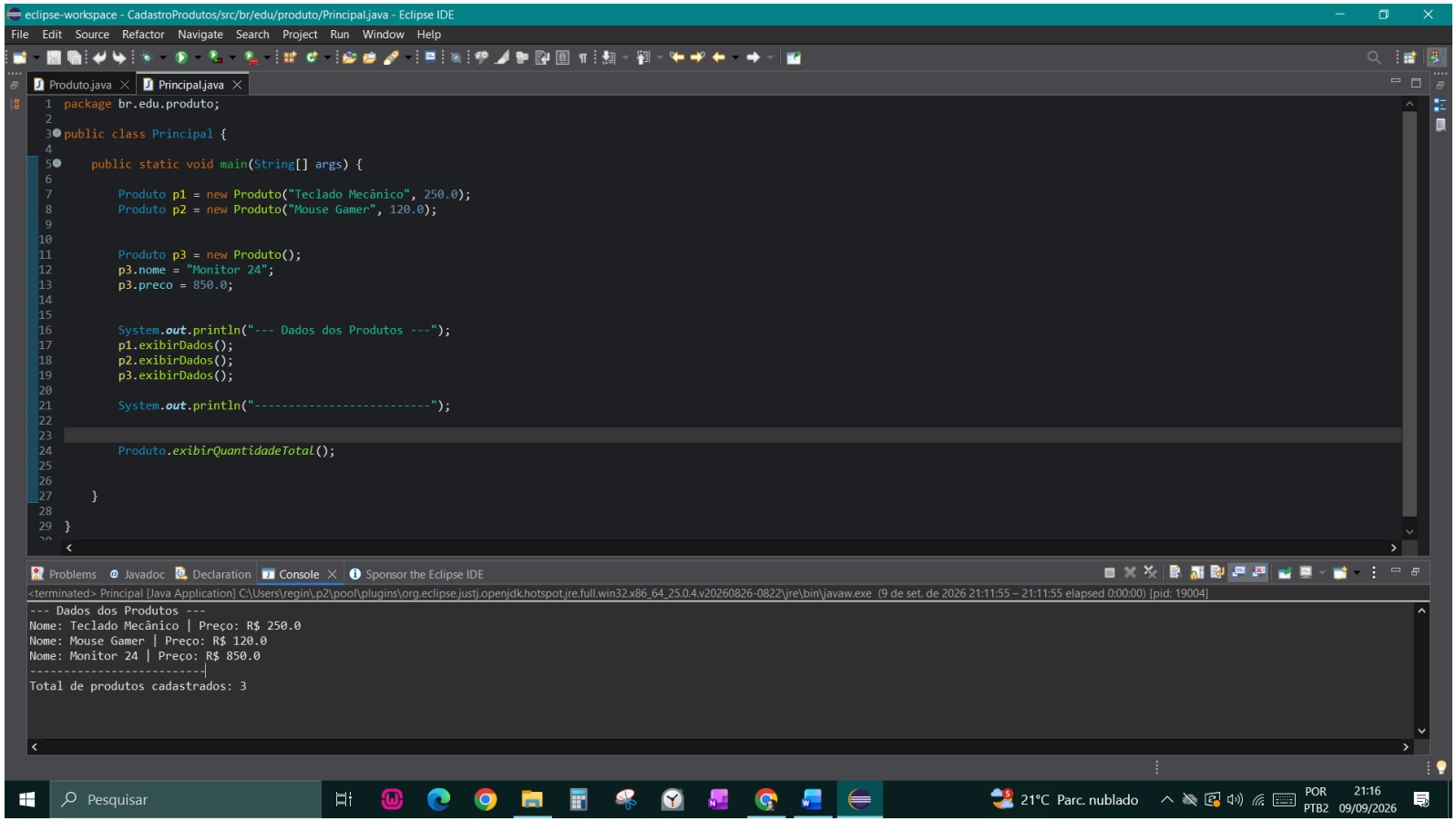
A variável quantidadeTotal recebeu o modificador static, o que vincula o atributo diretamente à classe e garante que todos os objetos instanciados compartilhem a mesma referência de memória para a contagem global dos itens cadastrados. Em seguida, implementou-se a sobrecarga de construtores (*overloading*) para proporcionar flexibilidade na criação dos objetos: o construtor padrão, sem parâmetros, limita-se a incrementar o contador global (quantidadeTotal++), enquanto o construtor parametrizado recebe o nome e o preço do produto, atribui esses valores às variáveis de instância por meio da palavra-chave this e também atualiza o contador de produtos.

Além disso, definiu-se o método exibirQuantidadeTotal() como estático (static), permitindo que a quantidade total de produtos cadastrados seja consultada diretamente através da classe (Produto.exibirQuantidadeTotal()), sem a necessidade de instanciar um objeto específico. Por fim, na classe Principal, estruturou-se o método main para simular a execução do sistema, onde foram instanciados três objetos com as diferentes modalidades de construtores e acionados os métodos para exibição dos dados e consulta do totalizador.

4. RESULTADO

Na Imagem 1, está apresentada a evidência de saída obtida no console do Eclipse IDE durante a execução da classe Principal. A execução demonstra o cadastro de três produtos e a contabilização exata pelo membro estático.

Imagem 1- Print do sistema em execução



4.1 Arquivo Produto.java

Código-fonte responsável por definir a estrutura, construtores e comportamentos da classe Produto:

```
public class Produto {


    String nome;
    double preco;
    static int quantidadeTotal = 0;


    public Produto() {
        quantidadeTotal++;
    }


    public Produto(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
        quantidadeTotal++;
    }


    public void exibirDados() {
        System.out.println("Nome: " + nome + " | Preço: R$ " + preco);
    }


    public static void exibirQuantidadeTotal() {
        System.out.println("Total de produtos cadastrados: " + quantidadeTotal);
    }
}
```

4.2 Arquivo Principal.java

Código-fonte responsável pelo fluxo principal de execução e teste das funcionalidades:

```
package br.edu.produto;

public class Principal {

    public static void main(String[] args) {

Produto p1 = new Produto("Teclado Mecânico", 250.0);
Produto p2 = new Produto("Mouse Gamer", 120.0);



Produto p3 = new Produto();
p3.nome = "Monitor 24";
p3.preco = 850.0;



System.out.println("--- Dados dos Produtos ---");
p1.exibirDados();
p2.exibirDados();
p3.exibirDados();

System.out.println("-------------------------");



Produto.exibirQuantidadeTotal();



    }

}
```

5. CONCLUSÃO

A realização desta aula prática, consolidou os conceitos teóricos de construtores, sobrecarga e membros estáticos na linguagem Java. A implementação da classe Produto e da classe Principal demonstrou a eficácia da sobrecarga de construtores, permitindo a criação de objetos tanto de forma genérica quanto com a inicialização direta de atributos. Além disso, a utilização do atributo e do método estático provou ser a solução ideal para gerenciar estados compartilhados globalmente na aplicação, garantindo o controle preciso do número de instâncias criadas. A atividade reforçou a importância do encapsulamento, do gerenciamento eficiente de memória e da organização estrutural em projetos Java.

6. REFERENCIAS

ALVES, William Pereira. Linguagem e Lógica de Programação. 1. ed. São Paulo: Editora Saraiva, 2013

FOWLER, Martin. UML essencial: um breve guia para a linguagem-padrão de modelagem de objetos. 3. ed. Porto Alegre: Bookman, 2011.

GAMMA, Erich et al. Padrões de projetos: soluções reutilizáveis de software orientados a objetos. Porto Alegre: Bookman, 2000.

MACHADO, Rodrigo Prestes; FRANCO, Marcelo Augusto; BERTAGNOLLI, Silvia de Castro. Desenvolvimento de software III: programação de sistemas web orientada a objetos em Java. Porto Alegre: SAGAH, 2016.

SCHILDT, Herbert. Java para Iniciantes. 6. ed. Porto Alegre: Bookman, 2015.

SILVA, Fernando M.; LEITE, M. C. D.; OLIVEIRA, Diego B. Paradigmas de programação. Porto Alegre: SAGAH, 2019.